

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

КУРСОВАЯ РАБОТА
по дисциплине «Программирование»
Тема: Обработка формата PNG

Студент гр. 5387

Степанов Л. А.

Преподаватель

Сычев А. Е.

Санкт-Петербург

2026

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ

Студент: Степанов Л. А.

Группа 5387

Тема работы: Обработка формата PNG

Исходные данные:

Программу требуется реализовать в виде утилиты, подобной стандартным linux-утилитам с которыми Вы уже имели дело: управление осуществляется посредством аргументов командной строки. Конкретный список команд зависит от решаемой задачи, но в общем случае обязательно:

1. Наличие справки, которая распечатывается при вызове утилиты без аргументов или стандартными ключами (-h, -help)
2. Обязательная обработка всех возможных исключительных ситуаций (даже бьющийся в агонии по клавиатуре пользователь не должен “уронить” вашу программу неверными аргументами)
3. Для всех(для которых это имеет смысл) ключей должны быть как полные так и сокращенные версии(-h, -help)
4. По-умолчанию последним аргументом утилите должно передаваться имя входного png файла. Имя выходного файла по умолчанию должно быть out.png, однако это должно быть возможным переопределить соответствующими ключом -o, -output
5. Утилита должна иметь функцию печати подробной информации о png-файле, ключ -info
6. В случае, если программой будут поддерживаться не все версии png-файлов, программа должна выводить об этом внятное сообщение, а не крашиться.
7. Программа за один запуск выполняет только одно действие (если программа может рисовать прямоугольник и круг, то за один запуск рисуется либо прямоугольник, либо круг)
8. Для каждого инструмента должен быть соответствующий ключ и ключи для его конфигурирования. Например, рисование прямоугольника может выглядеть как-то так:

Вариант 4.20

Программа должна иметь следующие функции по обработке изображений:

- (1) Рисование отрезка. Флаг для выполнения данной операции: -line.

Отрезок определяется:

- координатами начала. Флаг `–start`, значение задаётся в формате `x.y`, где `x` – координата по `x`, `y` – координата по `y`;
- координатами конца. Флаг `–end` (аналогично флагу `–start`);
- цветом. Флаг `–color` (цвет задаётся строкой `rrr.ggg.bbb`, где `rrr/ggg/bbb` – числа, задающие цветовую компоненту; пример `–color 255.0.0` задаёт красный цвет);
- толщиной. Флаг `–thickness`. На вход принимает число больше 0.

(2) Отражение заданной области. Флаг для выполнения данной операции: `–mirror`. Функциональность определяется:

- выбором оси отражения (горизонтальная или вертикальная). Флаг `–axis`, возможные значения `x` и `y`;
- координатами левого верхнего угла области. Флаг `–left_up`, значение задаётся в формате `left.up`, где `left` – координата по `x`, `up` – координата по `y`;
- координатами правого нижнего угла области. Флаг `–right_down`, значение задаётся в формате `right.down`, где `right` – координата по `x`, `down` – координата по `y`.

(3) Рисование пентаграммы в круге. Флаг для выполнения данной операции: `–pentagram`. Пентаграмма определяется:

- координатами центра и радиусом. Флаги `–center` (формат `x.y`: `x` – координата по оси `x`, `y` – по оси `y`) и `–radius` (число больше 0);
- толщиной линий и окружности. Флаг `–thickness`. На вход принимает число больше 0;
- цветом линий и окружности. Флаг `–color` (строка `rrr.ggg.bbb`; пример `–color 255.0.0` – красный цвет).

Содержание пояснительной записки:

«Содержание», «Введение», «Заключение», «Список использованных источников», «Приложение А. Пример работы программы».

Предполагаемый объем пояснительной записки:
Не менее 25 страниц.

Дата выдачи задания: 23.03.2025

Дата сдачи реферата 12.05.2025

Дата защиты реферата: 15.05.2026

Студент гр. 5387

Степанов Л.А.

Преподаватель

Сычев А. Е.

АННОТАЦИЯ

Данная курсовая работа посвящена разработке консольной утилиты для обработки растровых изображений в формате PNG на языке C. Программа реализована в стиле стандартных Linux-утилит с управлением через аргументы командной строки. Реализованный функционал включает: вывод информации о PNG-файле, рисование отрезков заданной толщины и цвета, зеркальное отражение прямоугольной области изображения, а также рисование пентаграммы в описанной окружности. При разработке использовалась библиотека libpng для чтения и записи PNG, динамическое выделение памяти для хранения пикселей, алгоритм Брезенхема для растеризации линий и геометрические вычисления для построения пентаграммы. Результаты подтверждают успешное выполнение всех поставленных требований.

SUMMARY

This course work is dedicated to the development of a command-line utility for processing PNG raster images in the C programming language. The program is implemented in the style of standard Linux utilities, controlled via command-line arguments. The implemented functionality includes: displaying PNG file information, drawing line segments with specified thickness and color, mirror-reflecting a rectangular region of an image, and drawing a pentagram inscribed in a circle. The development employed the libpng library for reading and writing PNG files, dynamic memory allocation for pixel data storage, Bresenham's line algorithm for rasterization, and geometric calculations for pentagram construction. The outcomes confirm successful fulfillment of all specified requirements.

СОДЕРЖАНИЕ

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ	2
АННОТАЦИЯ	5
SUMMARY.....	5
СОДЕРЖАНИЕ	6
ВВЕДЕНИЕ	7
1. ФУНКЦИИ	8
1.1 Функция main.c	8
1.2 Функция считывания текста	8
1.3 Функции выполняющие команды и вспомогательные функции	9
2. СТРУКТУРЫ	11
2.1 Структура Png.....	11
3. СБОРКА ПРОЕКТА.....	12
ЗАКЛЮЧЕНИЕ	13
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	14
ПРИЛОЖЕНИЕ А	15
ПРИМЕР РАБОТЫ ПРОГРАММЫ	15
ПРИЛОЖЕНИЕ Б	19
ИСХОДНЫЙ КОД ПРОГРАММЫ	19
Файл main.c:.....	19
Файл png_utils.c:	26
Файл png_utils.h:	38
Файл Make:.....	39

ВВЕДЕНИЕ

Целью работы является создание утилиты командной строки для обработки растровых изображений в формате PNG на языке программирования C.

В рамках работы решается ряд задач: реализация чтения и записи PNG-файлов с помощью библиотеки `libpng`; обработка аргументов командной строки в стиле стандартных Linux-утилит; рисование отрезков произвольной толщины и цвета с применением алгоритма растеризации; реализация зеркального отражения прямоугольной области изображения относительно горизонтальной или вертикальной оси; построение пентаграммы, вписанной в окружность заданного радиуса; вывод информации о структуре PNG-файла.

Для достижения поставленных целей используются механизмы динамического распределения памяти для хранения пиксельных данных, библиотека `libpng` для работы с форматом PNG, геометрические алгоритмы растеризации, а также модульная организация кода, разделяющая утилиты работы с PNG и логику основной программы по отдельным файлам.

1. ФУНКЦИИ

1.1 Функция `main.c`

Функция `main` является точкой входа утилиты. При запуске без аргументов она вызывает `print_help()` и завершается с кодом 0. В противном случае происходит разбор аргументов командной строки с помощью `getopt_long`.

Поддерживаются следующие действия, задаваемые флагами: `-i/--info` (вывод информации о PNG), `-l/--line` (рисование отрезка), `-m/--mirror` (зеркальное отражение области), `-p/--pentagram` (рисование пентаграммы в круге). Параметры действий передаются через флаги `--start`, `--end`, `--center`, `--radius`, `--axis`, `--left_up`, `--right_down`, `--thickness` и `--color`. Входной файл задаётся флагом `--input` или последним позиционным аргументом; выходной файл — флагом `-o/--output` (по умолчанию `out.png`).

После разбора аргументов функция открывает PNG-файл через `open_png()`. Значения толщины и цвета устанавливаются по умолчанию (толщина 1, цвет белый), если не были указаны явно. Затем выполняется соответствующее действие: для `--info` вызывается `print_png_info()`, для `--line` — `line()` и `save_png()`, для `--mirror` выполняется проверка обязательных параметров, затем вызываются `mirror()` и `save_png()`, для `--pentagram` проверяется наличие радиуса, после чего вызываются `pentagram()` и `save_png()`. По завершении память освобождается вызовом `free_png()`, и функция возвращает 0. При ошибках возвращается код 42.

1.2 Функция считывания текста

`open_file`

Функция `open_file` открывает файл по имени в режиме чтения («rb») и возвращает указатель на `FILE`. При неудаче выводит сообщение об ошибке в `stderr` и возвращает `NULL`.

`check_png`

Функция `check_png` проверяет, является ли открытый файл корректным PNG, считывая первые 8 байт сигнатуры и вызывая `png_sig_cmp`. Возвращает ненулевое значение, если файл является PNG.

`open_png`

Функция `open_png` загружает PNG-файл в структуру `Png`. Она открывает файл через `open_file`, проверяет сигнатуру через `check_png`, создаёт структуры `libpng` (`png_structp`, `png_infop`) и читает метаданные: ширину, высоту, глубину цвета, тип цвета и количество каналов. Выполняются преобразования: снятие 16-битной глубины до 8 бит, разворачивание палитры в RGB, расширение серого до RGB, добавление альфа-канала при наличии `tRNS`. Пиксельные данные сохраняются в динамически выделенном массиве `row_pointers`. При ошибке возвращается нулевая структура `Png`.

save_png

Функция `save_png` записывает изображение из структуры `Png` в файл. Открывает файл для записи, создаёт структуры записи `libpng`, устанавливает параметры IHDR и последовательно записывает строки пикселей. При ошибке выводит сообщение в `stderr`.

free_png

Функция `free_png` освобождает динамически выделенную память, занятую структурой `Png`: поочерёдно освобождает каждую строку пикселей, затем сам массив `row_pointers` и устанавливает указатель в `NULL`.

1.3 Функции выполняющие команды и вспомогательные функции

print_help

Функция `print_help` выводит в стандартный поток вывод справки по использованию утилиты: информацию о варианте курсовой работы, синтаксис вызова, список доступных действий и параметров с их кратким описанием. Вызывается при запуске без аргументов или при флаге `-h/--help`.

print_png_info

Функция `print_png_info` выводит основные характеристики загруженного PNG-изображения: ширину, высоту, глубину цвета, тип цвета и количество каналов. Принимает указатель на структуру `Png` и печатает поля через `printf`.

put_pixel

Функция `put_pixel` устанавливает цвет пикселя по координатам (x, y). Перед записью проверяет, что координаты не выходят за границы изображения. Записывает компоненты R, G, B в соответствующие байты строки пикселей; при наличии альфа-канала (4 канала) устанавливает его в 255.

draw_line

Функция `draw_line` рисует одиночную линию толщиной в один пиксель между двумя точками, используя алгоритм Брезенхема. Последовательно вычисляет шаги по осям и вызывает `put_pixel` для каждого пикселя вдоль отрезка.

dist1

Вспомогательная функция `dist1` вычисляет расстояние от точки (px, py) до отрезка, заданного двумя точками (x1, y1) и (x2, y2). Используется в функции `line` для определения принадлежности пикселя области отрезка заданной толщины.

line

Функция `line` рисует отрезок с заданной толщиной и цветом. Вычисляет ограничивающий прямоугольник вокруг отрезка с учётом толщины, затем для каждого пикселя в этом прямоугольнике вычисляет расстояние до отрезка через `dist1`. Если расстояние не превышает половины толщины, пиксель закрашивается вызовом `put_pixel`.

dist2

Вспомогательная функция `dist2` вычисляет евклидово расстояние между двумя точками. Используется в функции `circle`.

circle

Функция `circle` рисует окружность заданного радиуса с заданной толщиной линии. Для каждого пикселя в ограничивающем прямоугольнике вычисляется расстояние до центра; если оно близко к радиусу (в пределах половины толщины), пиксель закрашивается.

swap_pixels

Вспомогательная функция `swap_pixels` меняет местами два пикселя, представленных как массивы байт. Обменивает каждый канал по очереди через временную переменную.

mirror

Функция `mirror` выполняет зеркальное отражение прямоугольной области изображения. Принимает ось отражения ('x' — горизонтальное, 'y' — вертикальное), координаты углов области. Сначала нормализует координаты ($x1 \leq x2$, $y1 \leq y2$) и обрезает их до границ изображения. При оси 'x' попарно меняет пиксели левой и правой сторон каждой строки. При оси 'y' попарно меняет строки верхней и нижней частей области.

pentagram

Функция `pentagram` рисует пентаграмму, вписанную в окружность заданного радиуса с центром в (sx, sy). Сначала рисует окружность через `circle`. Затем вычисляет координаты пяти вершин, равномерно расположенных на окружности начиная с угла -90° . Соединяет вершины отрезками через одну (порядок $0 \rightarrow 2 \rightarrow 4 \rightarrow 1 \rightarrow 3 \rightarrow 0$), образуя пятиконечную звезду, вызывая `line` для каждой из пяти сторон звезды.

2. СТРУКТУРЫ

2.1 Структура Png

Структура Png определена в файле `png_utils.h` и служит для хранения всех данных о загруженном PNG-изображении в памяти. Она содержит следующие поля: `width` и `height` — ширина и высота изображения в пикселях; `color_type` — тип цвета PNG (например, RGB, RGBA, оттенки серого); `bit_depth` — глубина цвета в битах на канал; `channels` — количество каналов (3 для RGB, 4 для RGBA); `row_pointers` — указатель на массив указателей на строки пикселей, динамически выделяемый при загрузке изображения.

3. СБОРКА ПРОЕКТА

Makefile проекта расположен в корневой директории и обеспечивает автоматическую компиляцию исходных файлов. Все файлы с расширением .c в корневой директории подхватываются автоматически через подстановку \$(wildcard *.c), что позволяет добавлять новые модули без изменения Makefile. Заголовочные файлы расположены в директории include, которая передаётся компилятору через флаг -I\$(INCDIR).

Компиляция выполняется компилятором gcc со стандартом C11 и флагами предупреждений -Wall -Wextra. Для сборки конечного исполняемого файла sw используются флаги компоновщика -lpng (библиотека libpng) и -lm (математическая библиотека).

Поддерживаются следующие цели сборки:

1. make / make all — компилирует все .c файлы в объектные файлы (.o) и компоует исполняемый файл sw.
2. make clean — удаляет объектные файлы (.o) и исполняемый файл sw, позволяя выполнить полную пересборку с нуля.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана консольная утилита на языке C для обработки растровых изображений в формате PNG. Программа реализована в стиле стандартных Linux-утилит с управлением через аргументы командной строки и полной обработкой ошибочных ситуаций. Для достижения поставленной цели были применены следующие методы:

1. Реализация чтения и записи PNG-файлов с помощью библиотеки libpng с поддержкой различных типов цвета и автоматической нормализацией к формату RGB/RGBA.
2. Реализация алгоритмов геометрической растеризации: рисование отрезков с произвольной толщиной на основе вычисления расстояния от пикселя до отрезка, а также рисование окружностей через проверку расстояния до центра.
3. Реализация операции зеркального отражения прямоугольной области изображения относительно горизонтальной или вертикальной оси через попарный обмен пикселей.
4. Построение пентаграммы, вписанной в окружность: вычисление координат пяти вершин на окружности и соединение их в порядке, образующем пятиконечную звезду.
5. Организация модульной структуры проекта: логика работы с PNG вынесена в отдельные файлы png_utils.c/png_utils.h, а main.c содержит только разбор аргументов и управление потоком выполнения, что обеспечивает читаемость и расширяемость кода.

Ключевые результаты:

1. Программа корректно обрабатывает все предусмотренные действия: --info, --line, --mirror, --pentagram.
2. Все функции работы с PNG успешно реализованы и соответствуют требованиям задания.
3. Обеспечена модульность и надёжность программного кода.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Керниган Б., Ритчи Д. Язык программирования С. — 2-е изд. — М.: Вильямс, 2016. — 304 с.
2. ISO/IEC 9899:2011. Programming languages — C. — International Organization for Standardization, 2011.
3. Прата С. Язык программирования С. Лекции и упражнения. — 6-е изд. — М.: Вильямс, 2015. — 928 с.
4. GNU C Library Reference Manual. — URL: <https://www.gnu.org/software/libc/manual/> (дата обращения: 2025).

ПРИЛОЖЕНИЕ А

ПРИМЕР РАБОТЫ ПРОГРАММЫ

Ниже приведены примеры вызова утилиты `sw` для демонстрации каждого из поддерживаемых режимов работы. Команды выполняются в терминале Linux. Входной файл — `input.png`.

1. Вывод справки

```
./sw
```

или

```
./sw -h
```

Программа выводит справочную информацию с описанием всех доступных флагов и параметров.

Course work for option 4.20, created by Leonid Stepanov.

Usage: `sw [OPTIONS] [FILE]`

Действия:

<code>-i, --info</code>	Показать информацию о PNG
<code>-l, --line</code>	Нарисовать отрезок
<code>-m, --mirror</code>	Отразить область
<code>-p, --pentagram</code>	Нарисовать пентаграмму в круге

Параметры:

<code>-s, --start x.y</code>	Координаты начала
<code>-e, --end x.y</code>	Координаты конца
<code>-c, --center x.y</code>	Центр окружности
<code>-r, --radius N</code>	Радиус
<code>-a, --axis x y</code>	Ось отражения

-u, --left_up x.y Левый верхний угол

-d, --right_down x.y Правый нижний угол

-t, --thickness N Толщина линии

--color r.g.b Цвет в формате R.G.B

-o, --output FILE Выходной файл (по умолчанию out.png)

--input FILE Входной файл (или последний аргумент)

2. Вывод информации о PNG-файле (--info)

```
./cw --info input.png
```

Программа выводит ширину, высоту, глубину цвета, тип цвета и количество каналов изображения.

```
./cw --info input.png
```

Width: 330

Height: 247

Bit depth: 8

Color type: 6

Channels: 4

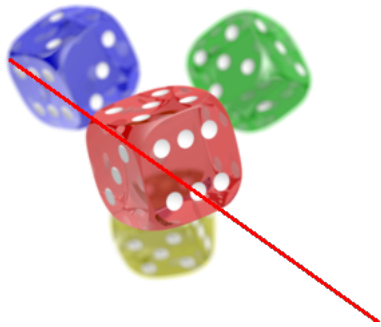
3. Рисование отрезка (--line)

```
./cw --line --start 50.50 --end 400.300 --color 255.0.0 --thickness 3  
-o out_line.png input.png
```

Рисует красный отрезок толщиной 3 пикселя от точки (50, 50) до точки (400, 300). Результат сохраняется в out_line.png.



[Рисунок 3 — Исходное изображение]



[Рисунок 4 — Результат --line]

4. Зеркальное отражение области (--mirror)

```
./cw --mirror --axis x --left_up 100.100 --right_down 400.400 -o
out_mirror.png input.png
```

Отражает прямоугольную область от (100, 100) до (400, 400) по горизонтальной оси (зеркало слева-направо). Результат сохраняется в out_mirror.png.



[Рисунок 5 — Результат --mirror --axis x]

```
./cw --mirror --axis y --left_up 100.100 --right_down 400.400 -o
out_mirror_y.png input.png
```

Отражает ту же область по вертикальной оси (зеркало сверху-вниз).

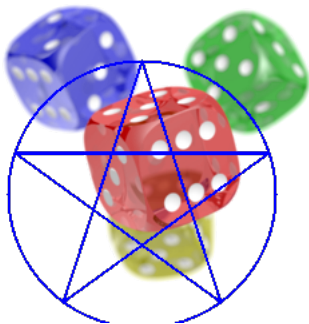


[Рисунок 6 — Результат --mirror --axis y]

5. Рисование пентаграммы в окружности (--pentagram)

```
./cw --pentagram --center 150.150 --radius 100 --color 0.0.255 --  
thickness 2 -o out_penta.png input.png
```

Рисует синюю пентаграмму с описанной окружностью радиуса 100 пикселей, центром в точке (150, 150), толщиной линий 2 пикселя. Результат сохраняется в out_penta.png.



[Рисунок 7 — Результат --pentagram]

6. Обработка ошибочных аргументов

```
./cw --line --start abc --end 400.300 input.png
```

При неверном формате аргумента программа выводит сообщение об ошибке в stderr и завершается с кодом 42.

```
./cw --line --start abc --end 400.300 input.png
```

Ошибка: Неверный формат --start

ПРИЛОЖЕНИЕ Б

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.c:

```
#include "png_utils.h"
#include <getopt.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void print_help()
{
    printf("Course work for option 4.20, created by Leonid Stepanov.\n");
    printf("Usage: cw [OPTIONS] [FILE]\n\n");
    printf("Действия:\n");
    printf("  -i, --info                Показать информацию о PNG\n");
    printf("  -l, --line                Нарисовать отрезок\n");
    printf("  -m, --mirror              Отразить область\n");
    printf("  -p, --pentagram           Нарисовать пентаграмму в круге\n\n");
    printf("Параметры:\n");
    printf("  -s, --start x.y           Координаты начала\n");
    printf("  -e, --end x.y             Координаты конца\n");
    printf("  -c, --center x.y          Центр окружности\n");
    printf("  -r, --radius N            Радиус\n");
    printf("  -a, --axis x|y            Ось отражения\n");
    printf("  -u, --left_up x.y         Левый верхний угол\n");
    printf("  -d, --right_down x.y      Правый нижний угол\n");
    printf("  -t, --thickness N         Толщина линии\n");
}
```

```

        printf("  --color r.g.b          Цвет в формате
R.G.B\n");
        printf(
            "  -o, --output FILE          Выходной файл (по
умолчанию out.png)\n");
        printf(
            "  --input FILE                Входной файл (или
последний аргумент)\n");
    }

    int main(int argc, char **argv)
    {
        if (argc == 1)
        {
            print_help();
            return 0;
        }

        opterr = 0;
        char *input_file = NULL;
        char *output_file = "out.png";
        int action = 0;

        int start_x = 0, start_y = 0, end_x = 0, end_y = 0;
        int center_x = 0, center_y = 0, radius = -1;
        char axis = 0;
        int lu_x = -1, lu_y = -1, rd_x = -1, rd_y = -1;
        int color_r = -1, color_g = -1, color_b = -1;
        int thickness = -1;

        struct option long_options[] = {
            {"help", no_argument, NULL, 'h'},
            {"info", no_argument, NULL, 'i'},
            {"line", no_argument, NULL, 'l'},
            {"mirror", no_argument, NULL, 'm'},
            {"pentagram", no_argument, NULL, 'p'},
            {"start", required_argument, NULL, 's'},
            {"end", required_argument, NULL, 'e'},

```

```

        {"center", required_argument, NULL, 'c'},
        {"radius", required_argument, NULL, 'r'},
        {"axis", required_argument, NULL, 'a'},
        {"left_up", required_argument, NULL, 'u'},
        {"right_down", required_argument, NULL, 'd'},
        {"thickness", required_argument, NULL, 't'},
        {"output", required_argument, NULL, 'o'},
        {"color", required_argument, NULL, 1000},
        {"input", required_argument, NULL, 1001},
        {NULL, 0, NULL, 0}};

    int opt;
    while ((opt = getopt_long(argc, argv,
"hilmps:e:c:r:a:u:d:t:o:",
                                long_options, NULL)) != -1)
    {
        switch (opt)
        {
            case 'h':
                print_help();
                return 0;
            case 'i':
                action = 1;
                break;
            case 'l':
                action = 2;
                break;
            case 'm':
                action = 3;
                break;
            case 'p':
                action = 4;
                break;
            case 'o':
                output_file = optarg;
                break;
            case 1001:
                input_file = optarg;

```

```

        break;
    case 's':
        if (sscanf(optarg, "%d.%d", &start_x,
&start_y) != 2)
        {
            fprintf(stderr, "Ошибка: Неверный
формат --start\n");
            return 42;
        }
        break;
    case 'e':
        if (sscanf(optarg, "%d.%d", &end_x,
&end_y) != 2)
        {
            fprintf(stderr, "Ошибка: Неверный
формат --end\n");
            return 42;
        }
        break;
    case 'c':
        if (sscanf(optarg, "%d.%d", &center_x,
&center_y) != 2)
        {
            fprintf(stderr, "Ошибка: Неверный
формат --center\n");
            return 42;
        }
        break;
    case 'u':
        if (sscanf(optarg, "%d.%d", &lu_x, &lu_y)
!= 2)
        {
            fprintf(stderr, "Ошибка: Неверный
формат --left_up\n");
            return 42;
        }
        break;
    case 'd':

```

```

        if (sscanf(optarg, "%d.%d", &rd_x, &rd_y)
!= 2)
        {
            fprintf(stderr, "Ошибка: Неверный
формат --right_down\n");
            return 42;
        }
        break;
    case 'r':
        radius = atoi(optarg);
        if (radius <= 0)
        {
            fprintf(stderr, "Ошибка: Радиус должен
быть > 0\n");
            return 42;
        }
        break;
    case 't':
        thickness = atoi(optarg);
        if (thickness <= 0)
        {
            fprintf(stderr, "Ошибка: Толщина
должна быть > 0\n");
            return 42;
        }
        break;
    case 'a':
        if (strcmp(optarg, "x") == 0) axis = 'x';
        else if (strcmp(optarg, "y") == 0) axis =
'y';
        else
        {
            fprintf(stderr,
                "Ошибка: Неверная ось
(используйте x или y)\n");
            return 42;
        }
        break;

```

```

        case 1000:
            if (sscanf(optarg, "%d.%d.%d", &color_r,
&color_g, &color_b) !=
                3)
            {
                fprintf(stderr, "Ошибка: Неверный
формат --color\n");
                return 42;
            }
            if (color_r < 0 || color_r > 255 || color_g
< 0 ||
                color_g > 255 || color_b < 0 || color_b
> 255)
            {
                fprintf(stderr,
                    "Ошибка: Значения цвета должны
быть 0-255\n");
                return 42;
            }
            break;
        case '?':
            fprintf(stderr, "Ошибка: Неизвестная
опция\n");
            return 42;
        }
    }

    if (!input_file)
    {
        if (optind < argc) input_file = argv[optind];
        else
        {
            fprintf(stderr, "Ошибка: Не указан входной
файл\n");
            return 42;
        }
    }

```

```

        if (action == 0)
        {
            fprintf(stderr,
                    "Ошибка: Не указано действие (например, -
i, -l, -m, -p)\n");
            return 42;
        }

        Png img = open_png(input_file);
        if (!img.row_pointers)
        {
            return 42;
        }

        if (thickness == -1) thickness = 1;
        if (color_r == -1)
        {
            color_r = 255;
            color_g = 255;
            color_b = 255;
        }
        int color[3] = {color_r, color_g, color_b};

        switch (action)
        {
            case 1:
                print_png_info(&img);
                break;
            case 2:
                line(&img, start_x, start_y, end_x, end_y,
color, thickness);
                save_png(output_file, &img);
                break;
            case 3:
                if (!axis || lu_x == -1 || rd_x == -1)
                {
                    fprintf(stderr, "Ошибка: Для отражения
требуется --axis, "

```

```

                                "--left_up,      --
right_down\n");
                                free_png(&img);
                                return 42;
                                }
                                mirror(&img, axis, lu_x, lu_y, rd_x, rd_y);
                                save_png(output_file, &img);
                                break;
                                case 4:
                                if (radius <= 0)
                                {
                                    fprintf(
                                        stderr,
                                        "Ошибка: Для пентаграммы требуется
указать --radius > 0\n");
                                    free_png(&img);
                                    return 42;
                                }
                                pentagram(&img, center_x, center_y, radius,
color, thickness);
                                save_png(output_file, &img);
                                break;
                                }

                                free_png(&img);
                                return 0;
                                }

```

Файл png_utils.c:

```

#include "png_utils.h"
#include <math.h>
#include <png.h>
#include <stdio.h>
#include <stdlib.h>

#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif
#define DEG_TO_RAD(x) ((x)*M_PI / 180.0)

```

```

#define CLAMP(v, min, max) ((v) < (min) ? (min) : ((v) >
(max) ? (max) : (v)))

FILE *open_file(char *filename)
{
    FILE *fp = fopen(filename, "rb");
    if (!fp)
    {
        fprintf(stderr, "Ошибка: Не удалось открыть файл
'%s'\n", filename);
    }
    return fp;
}

int check_png(FILE *fp)
{
    unsigned char header[8];
    if (fread(header, 1, 8, fp) != 8) return 0;
    return !png_sig_cmp(header, 0, 8);
}

Png open_png(char *filename)
{
    Png image = {0};

    FILE *fp = open_file(filename);
    if (!fp) return image;

    if (!check_png(fp))
    {
        fprintf(stderr, "Ошибка: Файл '%s' не является
PNG\n", filename);
        fclose(fp);
        return image;
    }

    png_structp png_ptr =

```

```

        png_create_read_struct(PNG_LIBPNG_VER_STRING,
NULL, NULL, NULL);
        png_infop info_ptr = png_create_info_struct(png_ptr);

        if (setjmp(png_jmpbuf(png_ptr)))
        {
            fprintf(stderr, "Ошибка при чтении PNG
структуры\n");
            png_destroy_read_struct(&png_ptr, &info_ptr,
NULL);
            fclose(fp);
            return image;
        }

        png_init_io(png_ptr, fp);
        png_set_sig_bytes(png_ptr, 8);
        png_read_info(png_ptr, info_ptr);

        image.width = png_get_image_width(png_ptr, info_ptr);
        image.height = png_get_image_height(png_ptr,
info_ptr);
        image.color_type = png_get_color_type(png_ptr,
info_ptr);
        image.bit_depth = png_get_bit_depth(png_ptr,
info_ptr);

        if (image.bit_depth == 16) png_set_strip_16(png_ptr);
        if (image.color_type == PNG_COLOR_TYPE_PALETTE)
            png_set_palette_to_rgb(png_ptr);
        if (image.color_type == PNG_COLOR_TYPE_GRAY &&
image.bit_depth < 8)
            png_set_expand_gray_1_2_4_to_8(png_ptr);
        if (png_get_valid(png_ptr, info_ptr, PNG_INFO_tRNS))
            png_set_tRNS_to_alpha(png_ptr);

        if (image.color_type == PNG_COLOR_TYPE_GRAY ||
            image.color_type == PNG_COLOR_TYPE_GRAY_ALPHA)
        {

```

```

        png_set_gray_to_rgb(png_ptr);
    }

    png_read_update_info(png_ptr, info_ptr);

    image.channels = png_get_channels(png_ptr, info_ptr);
    image.color_type = png_get_color_type(png_ptr,
info_ptr);

    image.row_pointers = (png_bytep
*)malloc(sizeof(png_bytep) * image.height);
    for (int y = 0; y < image.height; y++)
    {
        image.row_pointers[y] =
            (png_byte *)malloc(png_get_rowbytes(png_ptr,
info_ptr));
    }

    png_read_image(png_ptr, image.row_pointers);
    png_destroy_read_struct(&png_ptr, &info_ptr, NULL);
    fclose(fp);

    return image;
}

void save_png(const char *filename, Png *img)
{
    if (!img || !img->row_pointers) return;

    FILE *fp = fopen(filename, "wb");
    if (!fp)
    {
        fprintf(stderr, "Ошибка: не удалось открыть файл
%s для записи\n",
                filename);
        return;
    }
}

```

```

    png_structp png_ptr =
        png_create_write_struct(PNG_LIBPNG_VER_STRING,
    NULL, NULL, NULL);
    png_infop info_ptr = png_create_info_struct(png_ptr);

    if (setjmp(png_jmpbuf(png_ptr)))
    {
        fprintf(stderr, "Ошибка записи PNG\n");
        png_destroy_write_struct(&png_ptr, &info_ptr);
        fclose(fp);
        return;
    }

    png_init_io(png_ptr, fp);
    png_set_IHDR(png_ptr, info_ptr, img->width, img->
>height, 8, img->color_type,
        PNG_INTERLACE_NONE,
    PNG_COMPRESSION_TYPE_DEFAULT,
        PNG_FILTER_TYPE_DEFAULT);
    png_write_info(png_ptr, info_ptr);
    png_write_image(png_ptr, img->row_pointers);
    png_write_end(png_ptr, NULL);

    png_destroy_write_struct(&png_ptr, &info_ptr);
    fclose(fp);
}

void free_png(Png *img)
{
    if (!img || !img->row_pointers) return;
    for (int y = 0; y < img->height; y++)
    {
        free(img->row_pointers[y]);
    }
    free(img->row_pointers);
    img->row_pointers = NULL;
}

```

```

void print_png_info(Png *img)
{
    if (!img || !img->row_pointers) return;
    printf("Width: %d\n", img->width);
    printf("Height: %d\n", img->height);
    printf("Bit depth: %d\n", img->bit_depth);
    printf("Color type: %d\n", img->color_type);
    printf("Channels: %d\n", img->channels);
}

void put_pixel(Png *img, int x, int y, int color[3])
{
    if (x < 0 || x >= img->width || y < 0 || y >= img->
>height) return;

    png_bytep row = img->row_pointers[y];
    png_bytep pixel = &row[x * img->channels];

    pixel[0] = color[0];
    pixel[1] = color[1];
    pixel[2] = color[2];

    if (img->channels == 4)
    {
        pixel[3] = 255;
    }
}

void draw_line(Png *img, int x1, int y1, int x2, int y2,
int color[3])
{
    int dx = abs(x2 - x1);
    int dy = abs(y2 - y1);
    int sx = (x1 < x2) ? 1 : -1;
    int sy = (y1 < y2) ? 1 : -1;
    int err = dx - dy;

    while (1)

```

```

    {
        put_pixel(img, x1, y1, color);
        if (x1 == x2 && y1 == y2) break;
        int e2 = 2 * err;
        if (e2 > -dy)
        {
            err -= dy;
            x1 += sx;
        }
        if (e2 < dx)
        {
            err += dx;
            y1 += sy;
        }
    }
}

float dist1(float px, float py, float x1, float y1, float
x2, float y2)
{
    float vx = x2 - x1;
    float vy = y2 - y1;
    float wx = px - x1;
    float wy = py - y1;
    float c1 = wx * vx + wy * vy;
    if (c1 <= 0) return sqrtf(wx * wx + wy * wy);
    float c2 = vx * vx + vy * vy;
    if (c2 <= c1)
    {
        float dx = px - x2;
        float dy = py - y2;
        return sqrtf(dx * dx + dy * dy);
    }
    float b = c1 / c2;
    float bx = x1 + b * vx;
    float by = y1 + b * vy;
    float dx = px - bx;
    float dy = py - by;

```

```

        return sqrtf(dx * dx + dy * dy);
    }

    void line(Png *img, int x1, int y1, int x2, int y2, int
color[3], int thickness)
    {
        float radius = thickness / 2.0f;
        int minX = fmin(x1, x2) - thickness;
        int maxX = fmax(x1, x2) + thickness;
        int minY = fmin(y1, y2) - thickness;
        int maxY = fmax(y1, y2) + thickness;

        if (minX < 0) minX = 0;
        if (minY < 0) minY = 0;
        if (maxX >= img->width) maxX = img->width - 1;
        if (maxY >= img->height) maxY = img->height - 1;

        for (int y = minY; y <= maxY; y++)
        {
            for (int x = minX; x <= maxX; x++)
            {
                float d = dist1(x, y, x1, y1, x2, y2);
                if (d <= radius)
                {
                    put_pixel(img, x, y, color);
                }
            }
        }
    }

    float dist2(float x1, float y1, float x2, float y2)
    {
        float dx = x1 - x2;
        float dy = y1 - y2;
        return sqrtf(dx * dx + dy * dy);
    }

```

```

    void circle(Png *img, int cx, int cy, int radius, int
color[3], int thickness)
    {
        float t = thickness / 2.0f;
        int minX = cx - radius - thickness;
        int maxX = cx + radius + thickness;
        int minY = cy - radius - thickness;
        int maxY = cy + radius + thickness;

        if (minX < 0) minX = 0;
        if (minY < 0) minY = 0;
        if (maxX >= img->width) maxX = img->width - 1;
        if (maxY >= img->height) maxY = img->height - 1;

        for (int y = minY; y <= maxY; y++)
        {
            for (int x = minX; x <= maxX; x++)
            {
                float d = dist2(x, y, cx, cy);
                if (fabsf(d - radius) <= t)
                {
                    put_pixel(img, x, y, color);
                }
            }
        }
    }

    void swap_pixels(png_bytep a, png_bytep b, int channels)
    {
        png_byte tmp;
        for (int i = 0; i < channels; i++)
        {
            tmp = a[i];
            a[i] = b[i];
            b[i] = tmp;
        }
    }

```

```

void mirror(Png *img, char axis, int x1, int y1, int x2,
int y2)
{
    int channels = img->channels;
    if (x1 > x2)
    {
        int t = x1;
        x1 = x2;
        x2 = t;
    }
    if (y1 > y2)
    {
        int t = y1;
        y1 = y2;
        y2 = t;
    }
    x1 = CLAMP(x1, 0, img->width - 1);
    x2 = CLAMP(x2, 0, img->width - 1);
    y1 = CLAMP(y1, 0, img->height - 1);
    y2 = CLAMP(y2, 0, img->height - 1);

    if (axis == 'x')
    {
        for (int y = y1; y <= y2; y++)
        {
            png_bytep row = img->row_pointers[y];
            int left = x1;
            int right = x2;
            while (left < right)
            {
                swap_pixels(&row[left * channels],
&row[right * channels],
channels);
                left++;
                right--;
            }
        }
    }
}

```

```

        else if (axis == 'y')
        {
            int top = y1;
            int bottom = y2;
            while (top < bottom)
            {
                png_bytep row_top = img->row_pointers[top];
                png_bytep row_bottom = img->row_pointers[bottom];
                for (int x = x1; x <= x2; x++)
                {
                    swap_pixels(&row_top[x * channels],
                                &row_bottom[x * channels],
                                channels);
                }
                top++;
                bottom--;
            }
        }
    }

    void pentagram(Png *img, int cx, int cy, int radius, int
color[3],
                  int thickness)
    {
        circle(img, cx, cy, radius, color, thickness);
        int px[5], py[5];
        double start_angle = -90.0;

        for (int i = 0; i < 5; i++)
        {
            double angle = DEG_TO_RAD(start_angle + i * 72.0);
            px[i] = (int)round(cx + radius * cos(angle));
            py[i] = (int)round(cy + radius * sin(angle));
        }

        int current = 0;
        for (int i = 0; i < 5; i++)

```

```
    {  
        int next = (current + 2) % 5;  
        line(img, px[current], py[current], px[next],  
py[next], color,  
            thickness);  
        current = next;  
    }  
}
```

Файл png_utils.h:

```
#ifndef PNG_UTILS_H
#define PNG_UTILS_H

#include <png.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct
{
    int width, height;
    png_byte color_type;
    png_byte bit_depth;
    int channels;
    png_bytep *row_pointers;
} Png;

FILE *open_file(char *filename);
Png open_png(char *filename);
void save_png(const char *filename, Png *img);
void free_png(Png *img);

void print_png_info(Png *img);

void put_pixel(Png *img, int x, int y, int color[3]);

void draw_line(Png *img, int x1, int y1, int x2, int y2,
int color[3]);
void line(Png *img, int x1, int y1, int x2, int y2, int
color[3],
        int thickness);
void circle(Png *img, int cx, int cy, int radius, int
color[3], int thickness);

void mirror(Png *img, char axis, int x1, int y1, int x2,
int y2);
void pentagram(Png *img, int cx, int cy, int radius, int
color[3],
```

```
        int thickness);  
  
#endif
```

Файл Make:

```
TARGET = cw  
  
INCDIR = ./include  
  
CC = gcc  
  
CFLAGS = -I$(INCDIR) -Wall -Wextra -std=c11  
LDFLAGS = -lpng -lm  
  
SRCS = $(wildcard *.c)  
OBS = $(SRCS:.c=.o)  
  
all: $(TARGET)  
  
$(TARGET): $(OBS)  
    $(CC) $(OBS) -o $(TARGET) $(LDFLAGS)  
  
%.o: %.c  
    $(CC) $(CFLAGS) -c $< -o $@  
  
clean:  
    rm -f *.o $(TARGET)  
  
.PHONY: all clean
```